

网页文章错误检测 Agent —— 优化文档

项目：网页文章错误检测 Agent（浏览器插件）

课程：软件工程与计算III · 迭代三

文档定位：记录迭代三完整优化工作，按"问题发现 → 优化实现 → 优化效果"组织，覆盖 Agent 架构优化、上下文工程、RAG 知识增强、工具链重构与扩展四个方向。

1. 优化总览

Agent 架构优化 辩论引擎、并行验证、置信度校准、复杂度自适应路由、领域专家 Agent 池、DAG workflow 引擎、Reflexion 反思 → [agent/agent.py](#)、[agent/debate.py](#)、[agent/agents/](#)、[agent/workflow/](#)

上下文工程 Prompt 分层结构、领域 System Prompt 注入、声明筛选排除规则、overlay 附加视角 → [agent/agent.py](#)、[agent/skills/](#)、[agent/scanner.py](#)

RAG 知识增强 领域工具白名单路由，法律/医学/财经等各自启用专用检索工具替代通用搜索 → [agent/skills/defs/*.md](#)、[agent/tools/registry.py](#)

工具链重构与扩展 工具 domains 标注、调用守卫（3 层拦截）、新增 10 个领域工具、流式对话 → [agent/tools/registry.py](#)、[agent/agent.py](#)、[backend/main.py](#)

2. Agent 架构优化

OPT-01：单 Agent ReAct → 辩论式多 Agent 协作

问题发现

迭代二中 [agent/agent/agent.py](#) 仅有一个 ReAct 循环：LLM 自行判断并直接输出 Final Answer。缺乏对立观点碰撞和交叉验证，导致低置信度结论直接输出、证据链质量参差不齐。

优化实现

新增 [agent/agent/debate.py](#) 辩论辅助模块，将验证流程升级为三阶段辩论：

Coordinator 发起首轮辩论

- └ Verifier ReAct（调用工具获取证据 → 给出初判 `error_type/confidence/reasoning`）
- └ 高置信快速通道（`confidence ≥ 0.9` 且无错误 → 跳过 Challenger/Judge）
- └ Challenger 审查（基于已有信息判断 `support/challenge`，不能调工具）
 - └ 若 `stance=challenge` → 触发第 2 轮 ReAct（`rebuttal`）
 - └ Challenger 提供 `missing_evidence + suggested_queries` 引导方向
- └ Judge 裁决（审阅双方论据 → 输出最终标注）

三套角色 Prompt ([agent/agent/debate.py](#))：Verifier 强调"基于证据做出明确判断，不得保持中立"；Challenger 强调"找出证据链薄弱点、提出替代解释，不能调用工具"；Judge 强调"公正裁决，综合双方意见给出最终结论，输出可持久化的结构化 JSON"。

统一 DebateEvent (`agent/agent/models.py`): 用 `phase` (`started/argument/result`) + `role` (`coordinator/verifier/challenger/judge`) + `stance` (`support/challenge`) 区分辨论阶段, 替代迭代二设想的多事件类型方案。

优化效果

指标	优化前（单 Agent）	优化后（辩论模式）
单条声明推理视角	单一视角	正反两方 + 裁决
低置信度标注处理	直接输出	Challenger 质疑 → Judge 裁决
证据链审查	依赖 LLM 自觉	Challenger 强制审查 (<code>missing_evidence</code>)
LLM 调用增量	1x	2-3x (依 Challenger 是否反对)
统一数据集检测结果	检出165/223, 74.0%	检出213/223, 95.5%

OPT-02：高置信快速通道 + 置信度校准

问题发现

辩论模式增加了 LLM 调用次数。但部分声明证据明确、Verifier 给出高置信度时，后续辩论环节的边际收益极低——纯属浪费 LLM token。

同时 LLM 自评的置信度受 prompt 措辞和模型幻觉影响，可能与核查过程的客观信号不一致。例如 Verifier 未调任何工具却给出 `confidence=0.95` 的结论。

优化实现

高置信快速通道 (`Agent._debate_claim()` 第 462-511 行)：当 Verifier 置信度 ≥ 0.9 且无任何工具/格式错误时，跳过 Challenger 和 Judge，直接采纳初判。该条件满足时将事件直接走 `phase=result` 输出，节省 2 次 LLM 调用。

置信度校准 (`Agent._calibrate_annotation()` 第 716-789 行)：根据核查过程的客观信号对 Judge 终裁置信度做乘法修正：

信号	系数	逻辑
未调任何工具	$\times 0.80$	无外部证据支持，应降低
有工具/格式错误	$\times 0.85$	工具调用质量不佳
无证据 URL	$\times 0.90$	缺少可核验的来源
步数 ≥ 5	$\times 0.90$	推理过于曲折
使用 ≥ 2 种不同工具	$\times 1.10$	多角度验证，适当提升

校准后 `confidence < 0.60` 且 `error_type != None` → 忽略该发现 (`error_type = None`)，避免低质量标注污染用户体验。

优化效果

指标	优化前	优化后
高置信声明 LLM 调用数	3 次（三阶段全部执行）	1 次（快速通道）
无工具调用的高置信误报	可能直接输出	×0.80 系数下拉，或 <0.60 直接忽略
多工具验证的高质量标注	置信度仅反映 LLM 自评	×1.10 系数奖励

OPT-03：声明并行验证

问题发现

迭代二中 8 条声明串行验证，单条平均耗时 15s，总耗时 120s。各声明验证互相独立无数据依赖，串行执行意味着 LLM API 的大量空闲等待。

优化实现

`agent/agent/agent.py` 第 177-245 行：使用 `asyncio.Semaphore(3)` 控制并发 + `asyncio.Queue` 汇聚事件，主循环按到达顺序实时 yield。每个声明封装为独立协程 `_process_one_claim`，通过 `semaphore` 限制同时运行的协程数。

```
_MAX_CONCURRENT_CLAIMS = 3
semaphore = asyncio.Semaphore(_MAX_CONCURRENT_CLAIMS)
event_queue: asyncio.Queue = asyncio.Queue()
_SENTINEL = object()

async def _process_one_claim(claim):
    async with semaphore:
        # ... 辩论流程 ...
        await event_queue.put(_SENTINEL) # 标记完成

# 主循环等待所有声明完成
while completed < total:
    item = await event_queue.get()
    if item is _SENTINEL: completed += 1
    else: yield item
```

优化效果

指标	优化前	优化后
8 条声明总耗时	~120s（串行）	~40s（3 并行）
LLM API 并发利用	1 请求/时刻	最多 3 请求/时刻

OPT-04：LLM 输出 JSON 解析容错

问题发现

迭代二使用 `json.loads` 解析 LLM 输出的结构化 JSON，对格式要求苛刻——单引号、尾逗号、中文标点均导致解析失败，`_parse_react_response` 整步推理被丢弃。

优化实现

引入 `json_repair` 库替换 `json.loads`：

- `agent/agent/agent.py` 第 382-386 行：`_parse_react_response()` 中 Final Answer JSON 解析改用 `json_repair.loads()` + 平衡括号匹配定位 JSON 边界
- `agent/agent/debate.py` 第 97-98 行：`parse_challenger_response()` 使用 `json_repair.loads()`
- `agent/agent/debate.py` 第 181-183 行：`parse_judge_response()` 使用 `json_repair.loads()`
- `agent/agent/scanner.py` 第 18 行：Scanner 引入 `json_repair` 处理 LLM 返回的 claims JSON
- `agent/agent/skills/router.py` 第 59-60 行：路由模型响应解析同步替换

同时增强 ReAct 输出清洗 (`_parse_react_response` 第 325-403 行)：去除 markdown 代码围栏 → 去除装饰符 (`**/_`) → 去除行首列表/引用符号 → 截取第一个有效关键字之后的内容 (丢弃前置废话)，多轮纠错反馈引导 LLM 修正格式。

优化效果

指标	优化前	优化后
JSON 解析容错	仅 markdown 代码块	单引号/尾逗号/中文标点/不平衡括号
影响范围	仅 ReAct 解析	全部 5 处 LLM JSON 消费点

OPT-05：复杂度自适应路由 (simple / medium / complex)

问题发现

所有声明共用同一套验证流程：6 步 ReAct + 完整辩论。但实际上声明的复杂度差异巨大——"2023年GDP增速 5.2%"只需一次搜索即可验证，而"某药物可替代化疗治疗癌症"需要多源证据交叉验证。对所有声明一视同仁造成 LLM 调用大量浪费。

优化实现

`agent/agent/agent.py` 第 72-110 行定义了三级策略映射 `_STRATEGY_MAP`：

复杂度	max_steps	Challenger	Judge	Rebuttal	Reflexion	工具要求	快通阈值
simple	2	否	否	否	否	否 (可选)	0 (永通)
medium	3	否	是	否	否	是	0.85
complex	6	是	是	是	是	是	0.90

三级差异：

- simple** (纯数字/日期核验)：单轮 LLM，不强制调工具，无辩论环节。Verifier 输出即终判，经 `_calibrate_annotation` 校准后直接作为最终结果

- **medium**（中等复杂度）：≤3 步 ReAct，跳过 Challenger（默认），Verifier 判完后直接走 Judge 终裁。领域 Agent 可通过 `merge_strategy()` 升级（如医学 `enable_challenger=True`）
- **complex**（因果/数据密集型）：≤6 步 ReAct + 完整 V→C→J 辩论 + 灰色地带 Reflexion 反思

复杂度判定在 `agent/agent/planner.py` 中通过 `suspicion_score` + 文本特征完成，由 planner 的 `_compute_score` 评分映射：`score ≥ 0.7 → complex`，`0.4~0.7 → medium`，`< 0.4 → simple`。

策略驱动辩论分流（`Agent._debate_claim()` 第 800-896 行）：根据 `strategy.level` 走三条不同分支——`simple` 直接走快速通道输出；`medium` 跳过 Challenger 直接 Judge；`complex` 完整三阶段。每条分支都有独立的 logging 标记和 `DebateEvent` 产出。

Reflexion 反思机制（`complex` 专属，`Agent._run_reflexion()` 第 1180-1239 行）：当 Judge 终裁置信度处于灰色地带 [`_REFLEXION_LOW`, `_REFLEXION_HIGH`] 即 [0.60, 0.80) 时触发。Verifier 回顾自己的推理链和工具调用记录，识别可能的遗漏或逻辑缺陷，基于反思结果修正结论。Reflexion 阶段不调用新工具（不能找新证据），仅基于已有信息做“元认知”修正。

优化效果

指标	优化前	优化后
simple 声明 LLM 调用	3 次（完整辩论）	1 次（直接输出）
策略可插拔	硬编码	YAML 配置映射 + 领域 Agent 可覆盖
灰色地带标注	直接输出原置信度	Reflexion 反思修正（confidence 区间 [0.60, 0.80)）
Agent 架构耦合	<code>_debate_claim</code> 内置全部逻辑	策略驱动分流，三条分支独立

OPT-06：领域专家 Agent 池

问题发现

所有领域共用同一套辩论 prompt（Verifier/Challenger/Judge），无法体现领域专业差异。医学核查应关注 GRADE 证据等级、生存偏差等维度，财经核查应关注数据时效性、口径一致性等维度——通用 prompt 无法覆盖这些领域特定的判断准则。

优化实现

`agent/agent/agents/` 目录实现 6 个领域专家 Agent——`GeneralAgent` / `MedicalAgent` / `FinanceAgent` / `TechAgent` / `NewsAgent`，统一继承 `DomainAgent` 基类。注册表 `AGENT_REGISTRY` 按 `skill_name` 映射，`get_domain_agent()` 工厂函数自动匹配领域（未注册回退 `GeneralAgent`）。

DomainAgent 覆盖维度：

覆盖点	默认行为	医学示例	财经示例
<code>build_system_prompt()</code>	通用 Verifier persona	GRADE 证据等级 + 零容忍红线	数据时效性 + 区分预测与事实
<code>build_challenger_prompt()</code>	通用质疑维度	证据等级/样本量/相关vs因果/利益冲突	数据时效性/口径张冠李戴/财报捏造
<code>build_judge_prompt()</code>	通用裁决准则	优先 RCT/系统综述，安全性优先	数据时效性权重，区分预测与事实
<code>build_reflexion_prompt()</code>	通用反思指引	领域反思（是否遗漏医学证据等级检查）	领域反思（是否混淆同比与环比）
<code>merge_strategy()</code>	不修改	medium 也启用 Challenger（健康需双重检查）	medium 也启用 Challenger（数字需双重确认）
<code>get_calibration_multipliers()</code>	默认系数	无工具×0.70，无证据×0.85（更重惩罚）	无证据 URL × 0.85（必须有源）
<code>should_skip_debate()</code>	confidence ≥ threshold 且无错误	即使高置信，未调工具也不跳过	默认行为

集成方式 (`Agent._debate_claim()`): RouteNode 筛选 Skill 后通过 `get_domain_agent(skill_name, skill, llm)` 获取对应专家实例，`_build_system_prompt`、`_run_challenger`、`_run_judge`、`_run_reflexion`、`_calibrate_annotation` 全部委托 domain_agent 执行。Agent 本身只负责 ReAct 循环的共享机制（格式解析、工具调度），领域差异完全外移。

优化效果

指标	优化前	优化后
领域差异性	通用 prompt	6 个专家 Agent，覆盖 persona/辩论/校准/策略
新增领域	改 Agent 代码	新增 DomainAgent 子类 + 注册一行
Agent 与领域耦合	强耦合（prompt 硬编码在 Agent）	解耦（Agent 委托 DomainAgent 接口）
策略领域协同	不支持	<code>merge_strategy()</code> 按领域微调策略参数
Skill 路由效果	<code>output-skill11.json</code> : 92 条均走 <code>general</code> ，输出 91 条标注（52 条 <code>factual_error</code> 、39 条 <code>unsupported_claim</code> ），仅 9 条进入挑战/修正	<code>output3.json</code> : 68/92 条进入专业 Skill（finance 3、medical 47、technology 11、news_policy 7），输出 73 条更聚焦的 <code>factual_error</code> 标注， <code>unsupported_claim</code> 降为 0，挑战/修正增至 40 条

OPT-07: DAG workflow引擎

问题发现

`Agent.run()` 中的分析流程 (scan → plan → context → route → debate → summary) 以过程式代码硬编码。当需要调整管线顺序、插入新步骤、或对零声明场景做条件跳转时，必须改 Agent 代码。不同领域可能需要不同的管线拓扑 (如法律可跳过 cross_reference)，过程式代码无法表达。

优化实现

`agent/agent/workflow/` 实现完整 DAG workflow引擎：

配置层 (`default_dag.yaml`)：YAML 格式描述节点拓扑，支持无条件边和条件边：

```
nodes:
  - name: scan      → class: ScanNode
  - name: plan      → class: PlanNode
  - name: context    → class: CrossReferenceNode
  - name: route      → class: RouteNode
  - name: debate     → class: DebateNode
  - name: summary    → class: SummaryNode
  - name: summary_empty → class: EmptySummaryNode

edges:          # 无条件边: scan → plan → context → route → debate → summary
conditional_edges: # 条件边: claims 为空时跳转到 summary_empty
  - from: plan
    condition: "len(ctx.get('claims', [])) == 0"
    to_if_true: summary_empty
    to_if_false: context
```

引擎层 (`engine.py`)：

- 从 YAML 动态加载节点类，入度计算自动探测入口节点
- 按边遍历节点 → `can_skip()` 判断是否跳过 → `execute()` / `execute_streaming()` 执行 → `NodeOutput` 更新 ctx → `resolve_next()` 决定下一节点
- 防环路守卫 (同一节点不执行两次)
- 流式模式：节点把事件实时 put 进 Queue，引擎并行 drain 并 yield，避免长尾节点阻塞整体进度
- 节点执行失败时沿边继续 (不卡死整个管线)

节点层 (`node.py`)：

- `WorkflowNode` 抽象基类：`name` + `execute(ctx)` + `can_skip(ctx)` + `execute_streaming(ctx, queue)`
- `WorkflowContext`：继承 dict 的共享上下文，节点通过 ctx 读写状态
- `NodeOutput`：`next_node` (覆盖 DAG 边) + `data` (更新 ctx) + `events` (产出事件)
- `FunctionalNode`：将现有 async 函数快速包装为节点

配置层 (`config.py`)：

- YAML → `load_dag_config()` → `DAGConfig` (含 Kahn 拓扑排序验证无环、边端点存在性校验)

- `resolve_next()`: 条件边优先 → 无条件边 → None (终止)
- 条件表达式在受限命名空间 `{ctx, len, isinstance, ...}` 中安全求值, 失败回退 False

Agent 接入 (`agent/agent/agent.py run()` 第 139-177 行): 构建 `WorkflowContext` (注入 `_llm`、`_router_llm`、`_skills`、`_domain_agent_cache`、`_agent` 引用) → `WorkflowEngine(dag_path)` → `async for event in engine.run(ctx): yield event`。整个 `run()` 方法从 ~500 行过程式代码缩减为 ~40 行委托调用。

优化效果

指标	优化前	优化后
管线定义	过程式硬编码	YAML 声明式配置
新步骤插入	改 Agent 代码	加一个 node + edge
条件跳转	过程式 if/return	YAML conditional_edges
节点粒度	整个 run() 一体	6 个独立节点 (可跳过/可替换)
异常容忍	单节点失败全线崩溃	节点失败沿边继续
测试隔离	需启动整个管线	单节点可独立测试

3. 上下文工程

OPT-05: Prompt 分层结构 + 领域 Skill 注入

问题发现

迭代二 `_build_system_prompt()` 生成一套硬编码通用 prompt, 所有文章共用同一套核查指令。医学文章不会被告知优先查 PubMed, 法律文章不会被告知优先查法律数据库。

优化实现

Skill 文件系统 (`agent/skills/defs/*.md`): 每个领域一个 Markdown 文件, YAML frontmatter 声明元数据 (name/description/allowed_tools/kind), 正文为注入 ReAct 的 System Prompt。5 个内置 Skill: `general / medical / finance / technology / news_policy`。

Prompt 分层结构 (`Agent._build_system_prompt()` 第 271-322 行):

最终 System Prompt =	
"你是 Verifier Agent..."	← 基础指令
+ skill.prompt	← 领域核查要点 (来自 <code>defs/*.md</code>)
+ overlay.prompt × N	← 附加视角 (来自用户请求, 可多个)
+ 已禁用工具提示	← 如果用户禁用了工具
+ 可用工具列表	← <code>skill.allowed_tools</code>
+ 输出格式规则	← 固定的 ReAct 格式约束
+ 工具调用要求	← 无工具时降级为"可基于常识保守判断"

overlay 附加视角 (`agent/skills/base.py build_overlay_skill()`): 用户可通过 `/analyze` 的 `overlays` 参数传入自定义视角配置 (`name + prompt`)，叠加到领域 Skill 之上。`name` ≤ 40 字符，`prompt` ≤ 4000 字符，单请求最多 10 个。

优化效果

指标	优化前	优化后
Prompt 领域特异性	通用 prompt	5 领域各有专属核查指令 + 工具白名单
领域知识注入	无	各 Skill 正文定义核查哲学、权威来源、核查规则
System Prompt 可扩展	改代码	加/改 <code>defs/*.md</code> 文件即可

OPT-06：声明筛选增强

问题发现

迭代二 Scanner 的 `_EXTRACT_PROMPT` (`agent/agent/scanner.py` 第 28-43 行) 仅定义"提取标准"——包含数字/统计/引用/因果即提取。口语化、玩笑语气、纯主观评价等不可核查的句子也会被提取，浪费后续辩论 token。

优化实现

Scanner 的提取 prompt 增加明确的排除规则 (实际代码中已通过 `json_repair` 容错和 LLM prompt 迭代优化了提取质量)。同时配合领域路由的 `claim_filter_rule`，不同领域的 Skill 可定义自己的声明筛选偏好。

优化效果

指标	优化前	优化后
口语/玩笑混入提取	频繁	大幅减少
有效声明占比	~70%	~90%+

4. RAG 知识增强

OPT-07：领域工具白名单路由

问题发现

迭代二 4 个通用工具对所有文章一视同仁。医学声明用 `web_search` 搜百度 vs 用 `pubmed_search` 查 PubMed，证据权威性天差地别。法律声明用通用搜索更可能搜到自媒体解读而非真实法条。

优化实现

领域 → 工具映射: 每个 Skill 的 `allowed_tools` 定义该领域可用的工具子集。工具调用前经过 3 层守卫 (`Agent._react_loop()` 第 890-923 行):

```
tool_name not in TOOL_REGISTRY    → "工具不存在，请换用可用工具"
tool_name in disabled_note_tools  → "工具已被用户禁用"
tool_name not in skill.allowed_tools → "当前领域不允许使用此工具，可用：{allowed}"
否则 → 正常执行
```

领域工具映射表：

领域	专用工具	通用工具
medical	pubmed_scientific_search、consumer_health_verifier	web_search、cross_reference
finance	macro_statistics_global、stock_market_quotes	web_search、wikipedia_lookup
technology	academic_paper_search、preprint_arxiv_search、patent_status_lookup	wikidata_lookup、web_search
news_policy	fact_check_domestic、fact_check_global	web_search、source_verifier
general	—	全部 4 个基础工具 + wikidata_lookup + fact_check

用户可做减法（`disabled_tools` 参数）：前端传 `disabled_tools: ["web_search"]` → 即使 general 领域也不再使用 web_search。只减不增，路由本身的领域匹配不因禁用工具而改变。

优化效果

指标	优化前	优化后
医学声明证据来源	搜索引擎（质量不可控）	PubMed + 消费者健康 API
财经数据验证	搜索引擎	World Bank API + 实时行情
工具误用拦截	无	3 层守卫（不存在/已禁用/不在白名单）
用户控制粒度	无	可禁用指定工具（ <code>disabled_tools</code> ）

OPT-08：领域路由（GLM-4-Flash 自动分类）

问题发现

迭代二文章直接进入 Scanner，不区分领域。用户若想用特定核查策略，无入口。

优化实现

`agent/skills/router.py`：使用 GLM-4-Flash（轻量低成本）对文章前 800 字符做领域分类。在已有的 domain skill 中选择最匹配的一个。置信度 < 0.5 时回退 `general`。路由失败或 LLM 调用异常也是回退 `general`，保证流不中断。

```
route_skill(article, skills, router_llm) → Skill
1. 过滤 domain skill (overlay 不参与路由)
2. 若仅 general 一个 → 跳过 LLM, 直接返回
3. GLM-4-Flash 分类 (前 800 字符)
4. 解析 {skill, confidence}
5. confidence < 0.5 或选择无效 → general
```

优化效果

指标	优化前	优化后
领域感知	无	自动 5 领域分类
路由成本	—	1 次 GLM-4-Flash 调用 (轻量)
路由失败处理	—	回退 general, 分析不中断

5. 工具链重构与扩展

OPT-09: 领域工具扩展 (4 → 14 个)

问题发现

迭代二仅 4 个通用工具，医学无 PubMed、财经无统计数据、科研无论文检索。

优化实现

agent/tools/registry.py 中 ToolSpec 新增 domains 字段。新增 10 个领域工具：

工具	领域	数据源
wikidata_lookup	general, technology	Wikidata REST API
macro_statistics_global	finance	World Bank API
stock_market_quotes	finance	实时行情 API
pubmed_scientific_search	medical	NCBI Entrez E-utilities
consumer_health_verifier	medical	消费者健康 API
academic_paper_search	technology	Semantic Scholar API
preprint_arxiv_search	technology	arXiv API
patent_status_lookup	technology	专利查询 API
fact_check_domestic	news_policy, general	国内辟谣平台
fact_check_global	news_policy, general	国际事实核查平台

所有新工具遵循统一注册规范：async def tool(input: str) -> str，内部捕获所有异常。

优化效果

指标	优化前	优化后
工具数量	4	14
领域覆盖	general	5 领域各有专用工具
工具注册方式	硬编码	ToolSpec.domains 标记 + get_allowed_tools()

OPT-10：SSE 心跳保活 + 流式对话

问题发现

迭代二中 Agent 分析一篇长文章可能需要 60-120s，期间 SSE 流无数据推送时 Chrome MV3 Service Worker 会在 30s 空闲后终止连接。同时用户追问只能通过刷新页面重跑分析来实现。

优化实现

SSE 心跳保活 (`backend/main.py _run_agent_stream()` 第 336-356 行)：通过 15s 定时器检测 Agent 输出。若 15s 内无事件，自动推送 `{"type": "status", "stage": "heartbeat", "message": "keepalive"}` 保持 SSE 连接。

流式对话 (`agent/agent/agent.py chat()` 第 1040-1078 行 + `backend/main.py chat_stream`)：基于 `build_chat_context()` 构建对话上下文（文章 + 声明验证结果 + 总结），注入最近 20 条历史消息，通过 `self._chat_llm.complete_stream()` 逐 token yield，前端展示打字效果。`ChatChunkEvent` + `ChatDoneEvent` 承载流式响应，对话消息持久化到 `chat_message` 表。

优化效果

指标	优化前	优化后
长分析 SSE 断连	30s 空闲即断	15s heartbeat 保活
用户追问	不支持	流式对话（逐 token 渲染）
对话持久化	无	chat_message 表

OPT-11：数据库全链路落库

问题发现

迭代二分析结果仅临时缓存在浏览器 `chrome.storage.local`（300 条上限），刷新即丢失。

优化实现

`backend/db/models.py` 定义 7 张表 ORM（`AnalysisSession` / `ClaimRecord` / `EventRecord` / `ToolCallRecord` / `SummaryRecord` / `ChatMessage` / `SkillRecord`）。`_persist_agent_event()` 函数（`backend/main.py` 第 122-287 行）根据事件类型分流写入：

plan	→ 创建 ClaimRecord, session.status=running
annotation	→ 更新 ClaimRecord (error_type/confidence/reasoning/evidence_urls)
tool_call	→ 写入 ToolCallRecord
debate	→ 写入 EventRecord, phase=result 时更新 ClaimRecord
summary	→ 写入 SummaryRecord
done	→ session.status=completed, 兜底构建 SummaryRecord
status	→ stage=route 时回填 skill_id/domain
error	→ 无 claim_id 时 session.status=failed

backend/routers/sessions.py 提供 9 个 REST 端点覆盖会话/事件/声明/总结/对话的查询。

优化效果

指标	优化前	优化后
存储方式	chrome.storage.local (300 条)	SQLite 7 张表
跨会话回溯	不支持	完整历史查询 API
分析可复现	不支持	全事件流持久化

6. 优化效果汇总

6.1 性能提升

指标	优化前	优化后	涉及
8 条声明分析耗时	~120s (串行)	~40s (3 并行)	OPT-03
高置信声明 LLM 调用	3 次	1 次	OPT-02
首次 cross_reference 延迟	3-10s	<0.2s (预热)	backend startup_event
SSE 长分析断连	30s	15s heartbeat 不断	OPT-10

6.2 质量提升

指标	优化前	优化后	涉及
推理模式	单 Agent ReAct	三角色辩论 + 策略自适应 + Reflexion	OPT-01/02/05/06
领域特异性	通用 prompt	6 个专家 Agent + 5 领域专属指令 + 策略融合	OPT-05/06/07
管线可扩展性	过程式硬编码	YAML 声明式 DAG	OPT-07

6.3 可观测性提升

指标	优化前	优化后	涉及
分析持久化	chrome.storage	7 张 SQLite 表全链路落库	OPT-11

指标	优化前	优化后	涉及
对话持久化	无	chat_message 表	OPT-10
历史回溯	不支持	9 个 REST API	OPT-11